

# Secure and efficient transciphering for FHE-based MPC

Diego F. Aranha, Antonio Guimarães, Clément Hoffmann, and Pierrick Méaux

Tuesday 16th September, 2025



# Fully Homomorphic Encryption (FHE)[1]





x

f(x)

Homomorphically computes f(·)

[1] C. Gentry: A Fully Homomorphic Encryption Scheme.

# Hybrid Homomorphic Encryption (HHE)[2]



x

f(x)



Homomorphically computes Dec(sk,·) to get x

Homomorphically computes f(·)

[2]Naehrig, Lauter, Vaikuntanathan: Can homomorphic encryption be practical?

# Hybrid Homomorphic Encryption (HHE)



sk

x

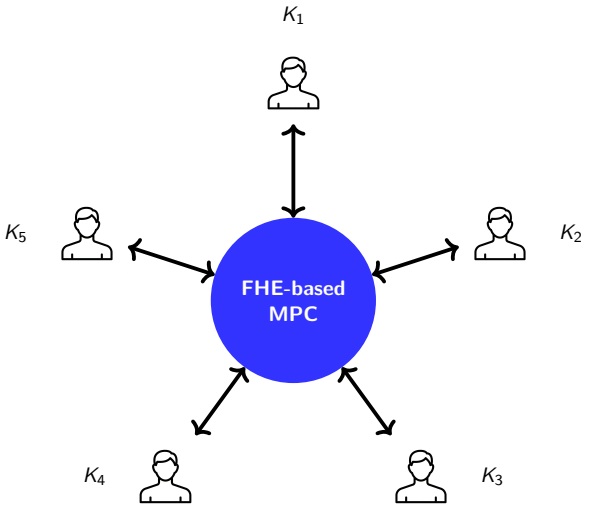
f(x)



Homomorphically computes $\mathsf{Dec}(\mathsf{sk},\cdot)$ to get x

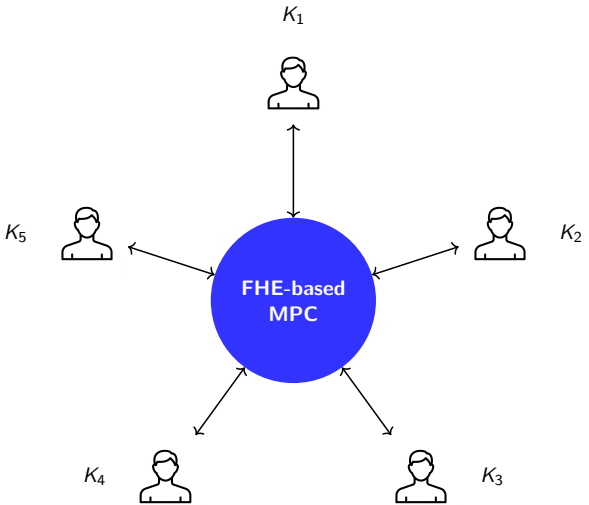
Homomorphically computes $f(\cdot)$

# MPC with FHE[3]



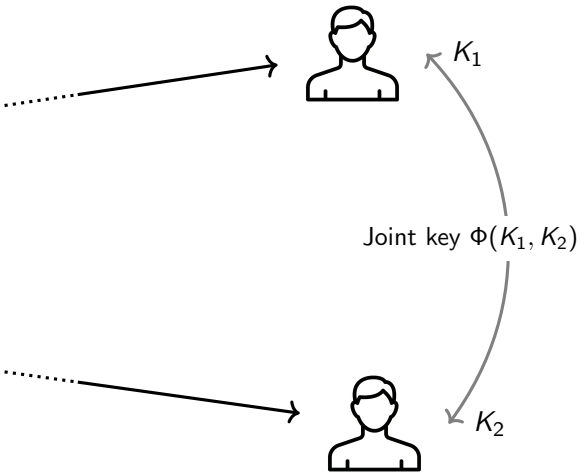

[3] N. Smart: Practical and efficient fhe-based MPC

# MPC with HHE[4]



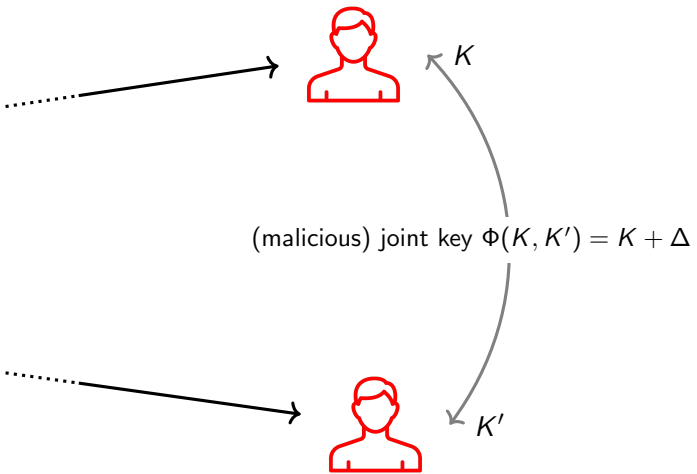

[4]N. Smart: Practical and efficient fhe-based MPC

# Related Key Attacks (RKA) security



# Related Key Attacks (RKA) security

**Related Key Attacks:** malicious adversaries generate symmetric key jointly and get access to *related key-streams*.



# Our contributions



- Formalize the use of transciphering in FHE-based MPC as a RKA problem

# Our contributions



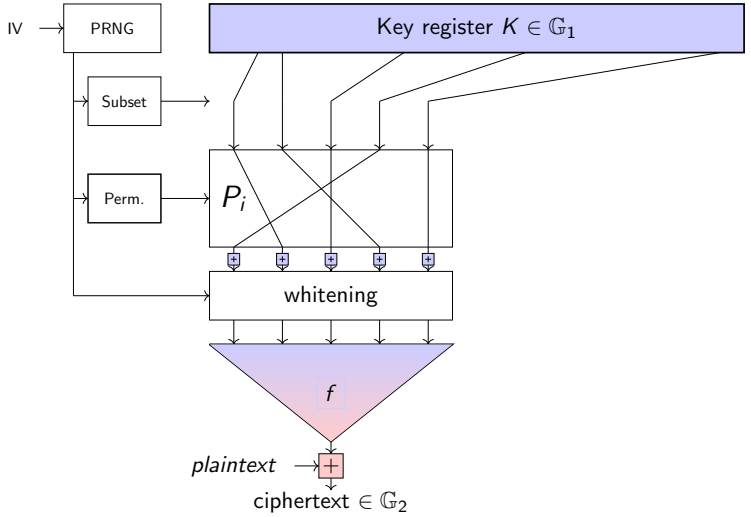
- Formalize the use of transciphering in FHE-based MPC as a RKA problem
- Proved that Margrethe is RKA resistant

# Our contributions



- Formalize the use of transciphering in FHE-based MPC as a RKA problem
- Proved that Margrethe is RKA resistant
- Efficient homomorphic implemention of Margrethe
  - Adapted our implementation method to the Elisabeth stream-cipher
  - Provided necessary algorithms to use our implementation in a MPC protocol

# Presentation of Margrethe - Mixed Filter Permutator




The Mixed Filter Permutator design

# Presentation of Margrethe[5]



- $\mathbb{G}_1 = \mathbb{F}_2$, $\mathbb{G}_2 = \mathbb{Z}_{16}$
- The filter is the direct sum of **14** identical **18**-variable inner functions
- The inner function is a **18-to-4** look-up table

[5] Hoffmann, Méaux, Standaert: The patching landscape of elisabeth-4 and the mixed filter permutator paradigm

# Presentation of Margrethe[5]

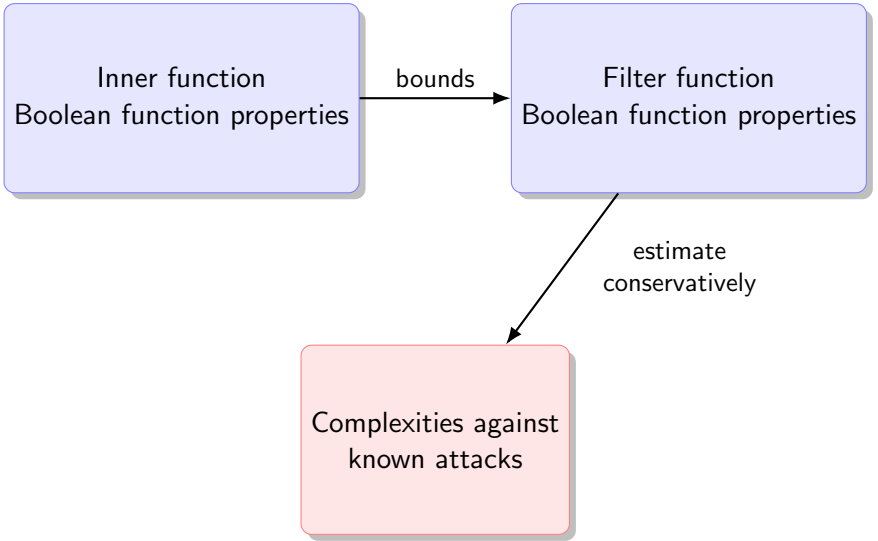


- $\mathbb{G}_1 = \mathbb{F}_2$, $\mathbb{G}_2 = \mathbb{Z}_{16}$
- The filter is the direct sum of **14** identical **18**-variable inner functions
- The inner function is a **18-to-4** look-up table

The only tricky operation to implement homomorphically is the *18-to-4 look-up table*.

[5] Hoffmann, Méaux, Standaert: The patching landscape of elisabeth-4 and the mixed filter permutator paradigm

# Security of Margrethe - Over $\mathbb{F}_2$



# Security of Margrethe - Over $\mathbb{Z}_{16}$



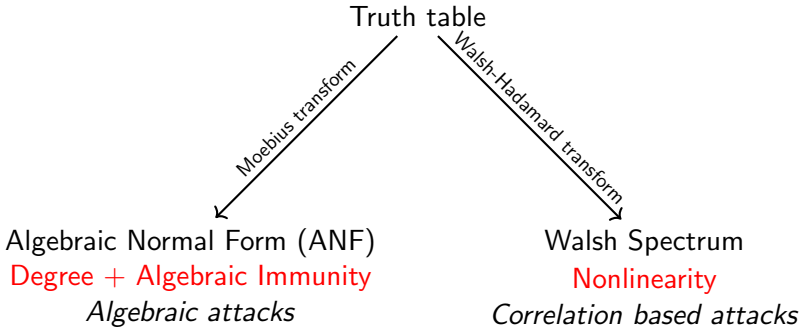
The probability for a random function $g : \mathbb{Z}_{16}^4 \longmapsto \mathbb{Z}_{16}$ to be a polynomial is $<< 2^{-128}$.[6]

- We verify that the inner function is not a polynomial.
- We use a code-based argument to justify that the filter does not share many output with a low degree function.

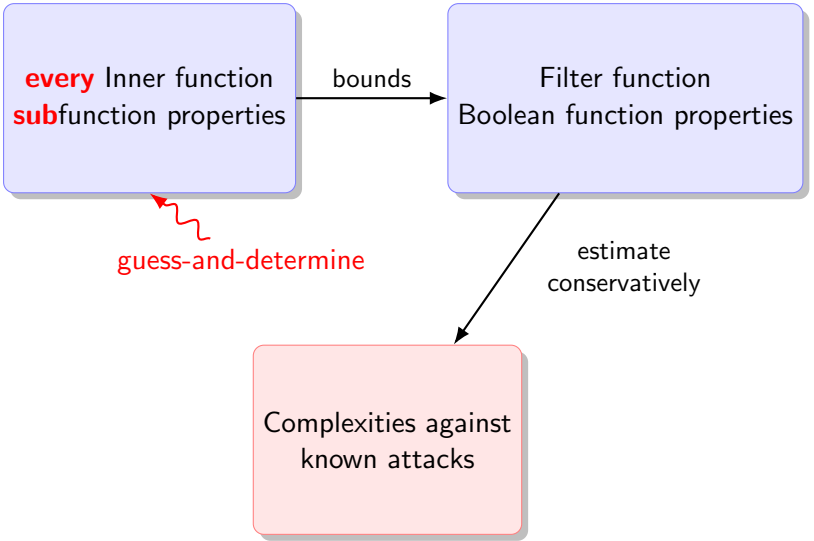
[6] Specker, Hungerbühler, Wasem. The ring of polyfunctions over $\mathbb{Z}/n\mathbb{Z}$

# Security of Margrethe - Over $\mathbb{F}_2$

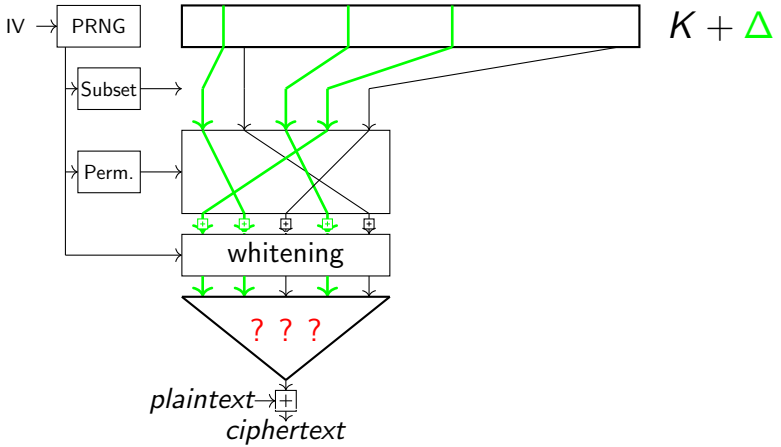
We study the inner function $g : \mathbb{F}_2^{18} \longmapsto \mathbb{F}_2^4$.



# Security of Margrethe - Guess-and-determine



# RKA security of Margrethe




Evaluating $KS(K + \Delta)$

# RKA security of Margrethe



**RKA security reduction**

The RKA security of Margrethe with filter function $f$ can be reduced to the regular analysis of Margrethe with filter function $f'$, where $f$ is a combination of symmetric functions and derivatives of $f$.

[7] Requires efficient implementation, specifically an AVX-512 Walsh-Hadamard transform

# RKA security of Margrethe



**RKA security reduction**

The RKA security of Margrethe with filter function $f$ can be reduced to the regular analysis of Margrethe with filter function $f'$, where $f$ is a combination of symmetric functions and derivatives of $f$.

To prove the RKA security :

1. $\forall \Delta$ compute[7] the new filter function $f'$
2. Run the security tests with $f'$

[7] Requires efficient implementation, specifically an AVX-512 Walsh-Hadamard transform

# RKA vs. Differential Fault Attacks (DFAs)



RKA security $\Longrightarrow$ security against DFAs

(with bitflips[8] in the key register)

[8] non-adaptive

# FHE toolbox

**Programmable Bootstrapping (PBS)**

Can be chained

Limited to small (negacyclic) look-up tables

# FHE toolbox



**Programmable Bootstrapping (PBS)**

Can be chained

Limited to small (negacyclic) look-up tables

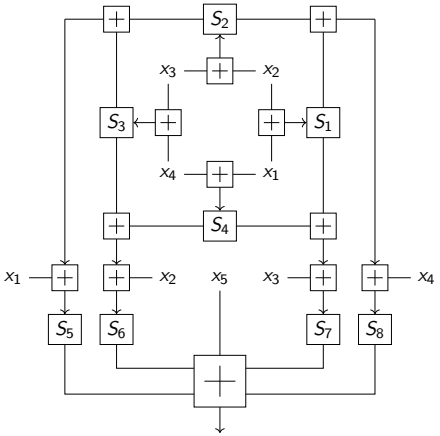
**Vertical Packing (VP)**

Efficiently evaluate big look-up tables

Expensive to evaluate two LUTs consecutively

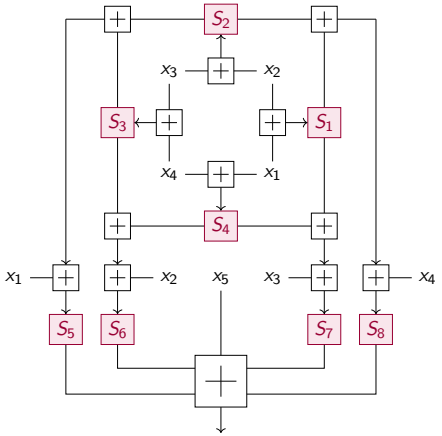
Bigger homomorphic representation of the inputs

# Elisabeth-4 inner function evaluation[9]



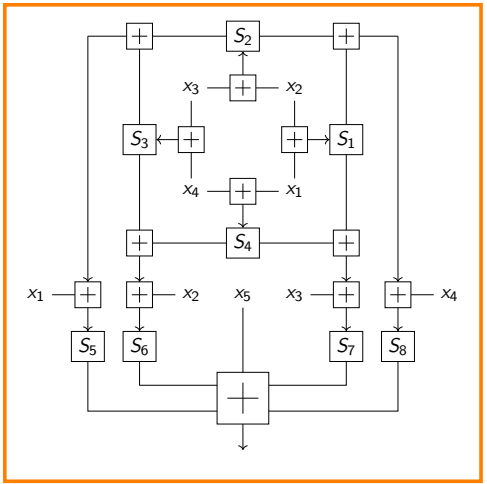

Elisabeth inner function

# Elisabeth-4 inner function evaluation



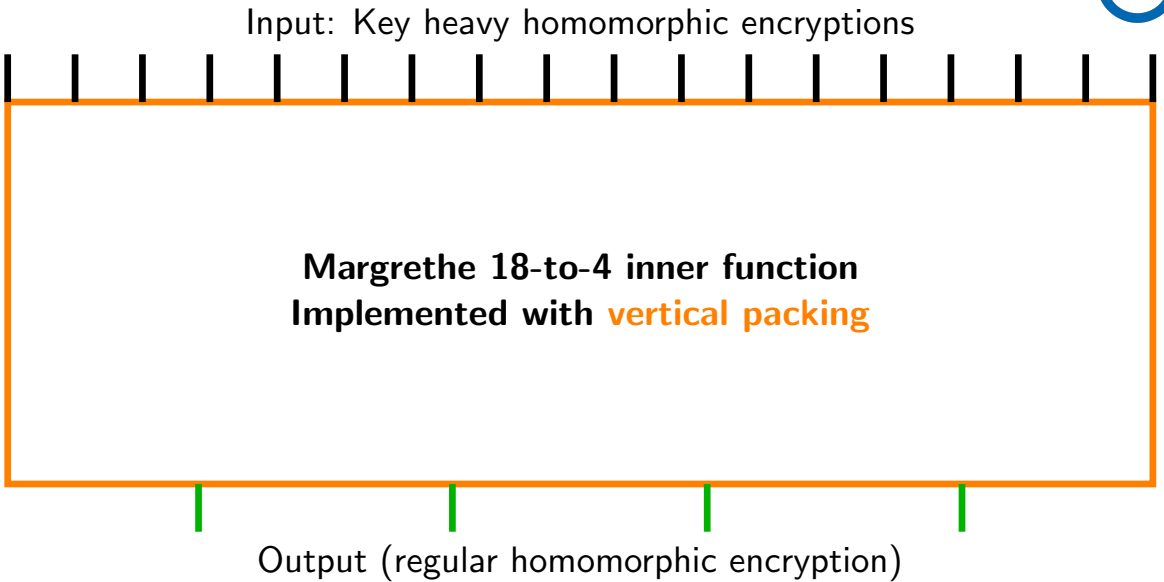

Elisabeth inner function - **PBS evaluation**

# Elisabeth-4 inner function evaluation

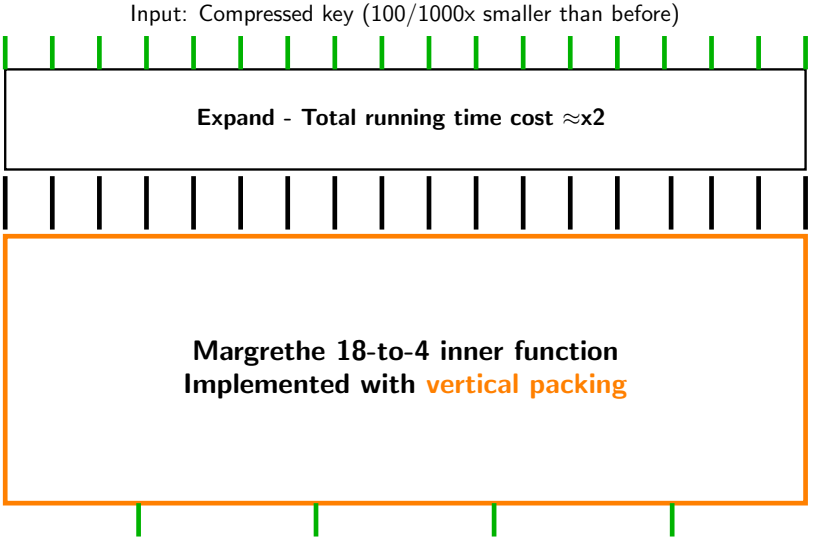



Elisabeth inner function - **VP evaluation**

# Margrethe implementation



# Compressed keys



# Other (important) details



- We adapt a known method[10] to handle plaintexts in a bigger space: **plaintext independent transciphering**.

[10] Méaux, Park, Pereira: Transciphering for FHE with Setup Independent of the Plaintext Space

# Other (important) details



- We adapt a known method[10] to handle plaintexts in a bigger space: **plaintext independent transciphering**.

- We describe and implements the algorithm necessary for mixing keys in the MPC protocol.

[10] Méaux, Park, Pereira: Transciphering for FHE with Setup Independent of the Plaintext Space

# Other (important) details



- We adapt a known method[10] to handle plaintexts in a bigger space: **plaintext independent transciphering**.
- We describe and implements the algorithm necessary for mixing keys in the MPC protocol.
- Our implementation is fully available

[10] Méaux, Park, Pereira: Transciphering for FHE with Setup Independent of the Plaintext Space

# Benchmark

| | Implement. | Single-thread | | Multi-thread | |
|---|---|---|---|---|---|
| | | Latency | Throughput | Latency | Throughput |
| FiLIP [MCJS19]] | [CDPP22] I | 2.62 | 381.68 | - | - |
| FiLIP [MCJS19] | [MPP23] I | 25.2 | 158.73 | - | - |
| FiLIP [MCJS19] | [MPP23] II | 71 | 56.50 | - | - |
| Elisabeth [CHMS22] | [CCH+24] | 2049 | 1.96 | - | - |
| Elisabeth-b [HMS23] | [CCH+24] | 5538 | 0.75 | - | - |
| Gabriel [HMS23] | [CCH+24] | 4662 | 0.86 | - | - |
| Kreyvium [CCF+18] | [CCH+24] | - | - | 134.00 | 7.47 |
| Kreyvium [CCF+18] | [BOS23] | - | - | 150.00 | 427.47[a] |
| Trivium [CP08] | [BOS23] | - | - | 121.00 | 529.47[a] |
| FRAST [CCH+24] | [CCH+24] | 6194.00 | - | - | 20.66[a] |
| **Margrethe** [HMS23] | **Set 1** | **27.2** | **147.06** | **3.12** | **1282.05** |
| **Margrethe** [HMS23] | **Set 2** | **54.2** | **73.8** | **4.21** | **950.11** |

[a] These implementations employ trivial parallelization to improve throughput.